TP 2 : Introduction de Threads en Java

Objectif : Ce TP vise à vous familiariser avec la création et la gestion de threads en Java en utilisant les classes `Thread` et `Runnable`, ainsi qu'à comprendre l'impact de la priorité et du nom du thread sur le temps d'exécution.

Description :

Exercice 1 : Création de Threads (Utilisation de `Thread`)

Instructions :

1. Créez une classe `ThreadTest`. Cette classe contient une méthode statique `Ecrire(String str)` qui permet d'écrire une chaine de caractères dans un fichier.
2. Créez une classe `AndroidThread` qui étend la classe `Thread`. Dans la méthode `run()`, faites en sorte que le thread Android écrive "I love Android" à l'aide de méthode `Ecrire` à chaque itération d'une boucle de 30000.
3. Créez une classe `PythonThread` qui étend également la classe `Thread`. Dans la méthode `run()`, faites en sorte que le thread Python écrive "I love Python" à l'aide de méthode `Ecrire` à chaque itération d'une boucle de 30000.
4. Dans `ThreadTest`, instanciez et démarrez ces threads dans la méthode `main`.

Exercice 1 (Suite) : Création de Threads (Utilisation de `Runnable`)

Instructions supplémentaires :

1. Créez une classe `RunnableTest`. Cette classe contient une méthode statique `Ecrire(String str)` qui permet d'écrire une chaine de caractères dans un fichier
2. Créez une nouvelle classe `AndroidRunnable` qui implémente l'interface `Runnable`. Implémentez la méthode `run()` pour que le thread Android écrive "I love Android" à l'aide de méthode `Ecrire`.
3. Créez une nouvelle classe `PythonRunnable` qui implémente également l'interface `Runnable`. Implémentez la méthode `run()` pour que le thread Python écrive "I love Python" à l'aide de méthode `Ecrire`.
4. Instanciez des objets de type `Thread` en passant les instances de `AndroidRunnable` et `PythonRunnable` comme arguments au constructeur.
5. Démarrez ces threads en utilisant `start()`.

Exercice 2 : Nom et Priorité des Threads

Instructions :

1. Dans la méthode `main`, attribuez le nom "Android Thread" au thread Android en utilisant `setName()`, et le nom "Python Thread" au thread Python.

2. Définissez la priorité maximale (MAX_PRIORITY) pour le thread Android et la priorité minimale (MIN_PRIORITY) pour le thread Python en utilisant `setPriority()`.
3. Exécutez à nouveau les threads.

Exercice 3 : Mesure du Temps d'Exécution

Instructions :

1. Dans chaque classe de thread (`AndroidThread` et `PythonThread` ou `AndroidRunnable` et `PythonRunnable` selon l'exercice), mesurez le temps d'exécution en millisecondes à l'aide de `System.currentTimeMillis()`.
2. À la fin de chaque thread, affichez le nom du thread et le temps d'exécution pris pour effectuer la tâche.
3. Exécutez les threads et observez les temps d'exécution affichés à la fin.

Consignes :

- Vous devez créer deux fichiers source distincts pour les classes `AndroidThread`, `PythonThread`, `AndroidRunnable`, et `PythonRunnable`.
- Assurez-vous que la méthode `utiliser_fichier` est correctement implémentée dans la classe `Main` pour éviter des erreurs de compilation.
- Vous pouvez utiliser le chemin de fichier spécifié dans la méthode `utiliser_fichier`, ou modifier le chemin pour correspondre à votre propre configuration.
- Les threads doivent effectuer 30 000 itérations de leur tâche respective.
- Assurez-vous que le fichier de sortie est correctement fermé après chaque utilisation.

Exercice : Compte rendu

1. Créez une classe `CommandProcessor` qui représente le traitement des commandes. Cette classe doit implémenter l'interface `Runnable`.
2. Dans la classe `CommandProcessor`, implémentez la méthode `run()`. Dans cette méthode, effectuez un traitement simulé de commandes pour un type de produit spécifique ("Produit A" ou "Produit B"). Pour simuler le traitement, utilisez une boucle pour effectuer un certain nombre d'itérations.
3. Créez deux instances de la classe `CommandProcessor` : une pour le "Produit A" et une pour le "Produit B".
4. Affectez un nom distinct à chaque instance en utilisant la méthode `setName()` de la classe `Thread`. Par exemple, donnez le nom "Thread Produit A" au thread correspondant au "Produit A", et "Thread Produit B" au thread correspondant au "Produit B".
5. Définissez la priorité des threads : donnez une priorité maximale (MAX_PRIORITY) au thread du "Produit A" et une priorité minimale (MIN_PRIORITY) au thread du "Produit B" en utilisant la méthode `setPriority()` de la classe `Thread`.
6. Démarrez les deux threads en utilisant la méthode `start()`.

7. Chaque thread effectuera un traitement simulé pendant un certain nombre d'itérations. Mesurez le temps total pris par chaque thread pour terminer son traitement en utilisant `System.currentTimeMillis()`.
8. Affichez le nom du thread et le temps pris à la fin de l'exécution de chaque thread.